

SLFD Specification

Simple Library Functions Define

Version 10.2

July 30, 2026

1 Knowledge

Welcome to Simple Library Functions Define (SLFD). This is an Interface Specification of E-comOS or other OSes that implement **SLFD**. Please note we support C language only for now, but we may support other languages in the future (not include C++, never). If you are a C++ coder, and you want to use SLFD, please use

```
extern "C" {  
    /* SLFD code here */  
}
```

to do your works.

2 Scope

This document defines the core operating system interface known as **SLFD** (Simple Library Functions Define). It provides system calls for process management, memory allocation, file/device I/O, pipes, network communications, event multiplexing (hear interface), inter-process communication (IPC), time operations, environment variables, file system operations, command execution, thread management, zero-copy I/O, asynchronous I/O, and additional features such as user/group management, shared memory, resource limits, and privilege elevation.

The interface is designed to be simple and extensible. Security policies (e.g., privilege levels, access control) are described in Appendix A (ULA - User Level Assessment) and are fully integrated into the core API using integer bitmasks.

All file system paths in this specification follow the SLFD FD layout defined in Appendix B, which includes top-level directories `/bin`, `/dev`, `/conf`, `/usr`, `/lib`, `/tmp`, `/sys`, and `/mnt`.

Kernel Architecture Note: This specification does not mandate a particular kernel architecture. Both monolithic kernels and microkernels can implement SLFD. The behaviour described for each function is mandatory; the underlying implementation may use direct trapping, user-space servers, or any combination thereof.

2.1 Design Philosophy

SLFD aims to provide a consistent, minimal, and secure interface. Key principles:

- **Separation of creation and execution:** Process objects are created with `proc_new` and later started with `proc_start`, allowing configuration in between.
- **Explicit resource declaration:** Processes must declare their needed capabilities via `proc_setflags` before starting (cannot be elevated later without `hlud`).
- **Unified I/O for local resources:** Files, devices, and pipes use `open/read/write/close`.
- **Dedicated network functions:** Network I/O uses separate calls (`net_connect`, etc.) to avoid confusion with local file semantics.
- **Event multiplexing as a first-class citizen:** The `hear` interface allows waiting on any event source (I/O, IPC, signals, process state).
- **ULA privilege model:** Four levels encoded as bitmasks, replacing traditional UID/GID/mode.t for better security and simplicity.

3 Head Files

Implementations shall provide the following header files. All headers are located under `/usr/include`.

3.1 `<slfd.h>`

Primary header, includes all other SLFD headers. Defines basic types, resource flags, signal numbers, I/O constants, and function prototypes.

3.2 `<slfd/ula.h>`

Defines ULA privilege levels and bitmask layout for file attributes.

```
#define LV_HIGHEST    0
#define LV_GENERAL    1
#define LV_LOW        2
#define LV_GETTING    3

#define ULA_CREATOR_SHIFT    24
#define ULA_CALLER_SHIFT    16
#define ULA_FLAG_EXEC        (1<<8)
#define ULA_FLAG_HIDDEN      (1<<9)
#define ULA_FLAG_ARCHIVE     (1<<10)
#define ULA_TYPE_FILE        (0<<4)
#define ULA_TYPE_DIR         (1<<4)
#define ULA_TYPE_LINK        (2<<4)
#define ULA_TYPE_DEV         (3<<4)

#define ULA_MAKE_ATTR(creator, caller, flags, type) \
    (((creator) << ULA_CREATOR_SHIFT) | ((caller) << \
        ULA_CALLER_SHIFT) | (flags) | (type))
```

3.3 <slfd/fs.h>

Extended file system operations (see Section ??).

3.4 <slfd/thread.h>

Threading primitives (see Section 12).

3.5 <slfd/performance.h>

Zero-copy and asynchronous I/O (implementation-defined).

3.6 <slfd/ipc_perf.h>

IPC performance tuning (implementation-defined).

3.7 <slfd/hear.h>

Event multiplexing (hear interface), see Section 10.

4 Terminology and Conventions

4.1 Types

```
typedef int fid_t;           /* file/device/pipe/network
    identifier */
typedef int pid_t;          /* process identifier */
typedef long off_t;         /* file offset (signed 64-bit) */
typedef long ssize_t;       /* signed size (64-bit) */
typedef unsigned long size_t; /* unsigned size (64-bit) */
typedef uint32_t thread_id; /* thread identifier for IPC and
    threading */

typedef struct {
    int argc;
    char **argv;
    char **envp;
} parg_t;

struct stat {
    size_t st_size;
    uint32_t st_attr_mask;
    /* other fields implementation-defined */
};

struct fs_dirent {
    char d_name[256];
    uint32_t d_type;
};
```

```

struct utsname {
    char sysname[32];
    char nodename[32];
    char release[32];
    char version[32];
    char machine[32];
};

struct rlimit {
    unsigned long rlim_cur;
    unsigned long rlim_max;
};

#define IPC_MAX_DATA_SIZE 4096
typedef struct ipc_message {
    uint32_t type;
    uint32_t source;
    uint32_t target;
    uint32_t timestamp;
    uint32_t size;
    uint32_t sequence;
    uint8_t data[IPC_MAX_DATA_SIZE];
} ipc_message_t;

```

4.2 Standard Constants

```

/* Resource flags (bitmask) */
#define PROC_IO_READ      (1<<0)
#define PROC_IO_WRITE    (1<<1)
#define PROC_NETWORK     (1<<2)
#define PROC_MEMORY      (1<<3)

/* Signal numbers (PROCSIG_* are aliases to traditional signal
   numbers) */
#define PROCSIG_HUP       1
#define PROCSIG_INT       2
#define PROCSIG_QUIT      3
#define PROCSIG_ILL       4
#define PROCSIG_TRAP      5
#define PROCSIG_ABRT      6
#define PROCSIG_BUS       7
#define PROCSIG_FPE       8
#define PROCSIG_KILL      9
#define PROCSIG_USR1     10
#define PROCSIG_SEGV     11
#define PROCSIG_USR2     12
#define PROCSIG_PIPE     13
#define PROCSIG_ALRM     14
#define PROCSIG_TERM     15
#define PROCSIG_CHLD     17
#define PROCSIG_STOP     18

```

```

#define PROCSIG_CONT      19
#define PROCSIG_TSTP      20

/* io_seek whence */
#define SEEK_SET  0
#define SEEK_CUR  1
#define SEEK_END  2

/* mount flags */
#define MS_RDONLY      1
#define MS_NOSUID      2
#define MS_NODEV      4
#define MS_NOEXEC      8
#define MS_SYNCHRONOUS 16
#define MS_REMOUNT     32
#define MS_MANDLOCK     64
#define MS_DIRSYNC     128
#define MS_NOATIME      256
#define MS_NODIRATIME  512
#define MS_BIND         1024
#define MS_MOVE         2048
#define MS_REC          4096
#define MS_SILENT       8192
#define MS_POSIXACL     16384
#define MS_UNBINDABLE   65536
#define MS_PRIVATE      131072
#define MS_SLAVE        262144
#define MS_SHARED       524288
#define MS_RELATIME     1048576
#define MS_KERNMOUNT    2097152
#define MS_I_VERSION    4194304
#define MS_STRICTATIME  8388608
#define MS_LAZYTIME     16777216

/* umask constants */
#define UMASK_DEFAULT  0022

```

4.3 Error Handling

Functions return an integer (`int` or `ssize_t`). Negative values indicate an error; the absolute value is one of the error codes defined in Section 13. Non-negative indicates success (or positive data like PID, `fid_t`, bytes transferred). On failure, the global `errno` is also set to the absolute value of the error code, allowing use of standard error-checking idioms. Callers may check either the return value or `errno`.

4.4 Conventions

- Strings are null-terminated.
- Passing `NULL` where not allowed yields `-EINVAL`.

- File descriptors 0, 1, and 2 are pre-opened upon process startup. Unless explicitly redirected by the caller, they are mapped to the process's controlling terminal or system console as device types ("d"). (*Updated in 10.3*)
- File attributes are integer bitmasks using ULA macros.

5 Process Management

5.1 `proc_new`

```
pid_t proc_new(const char *args, uint32_t privilege_mask);
```

5.1.1 Description

Creates a new process object and allocates a unique PID. The process does not start executing immediately; a subsequent call to `proc_start` is always required, regardless of whether `args` is "n" (new) or "s" (clone). This ensures a unified creation-startup flow.

5.1.2 Parameters

Parameter	Type	Description
<code>args</code>	<code>const char *</code>	Mode: "n" (new, no memory image) or "s" (clone, inherits a copy of parent's memory image).
<code>privilege_mask</code>	<code>uint32_t</code>	ULA privilege mask (see <code><slfd/ula.h></code>).

5.1.3 Return Value

- **Success:** Returns a positive PID.
- **Failure:** Returns a negative error code:
 - `-EINVAL`: Invalid `args` string.
 - `-ENOMEM`: Insufficient kernel memory for new process.
 - `-EPERM`: Caller lacks privilege to create process with the given `privilege_mask`.

5.1.4 Design Rationale

Traditional UNIX `fork()` clones and runs immediately. SLFD separates creation from starting to allow configuration (e.g., via `proc_setflags`) between them. Both "n" and "s" modes now require `proc_start` to run, making the flow uniform and predictable.

5.1.5 Notes for Callers

- The `privilege_mask` must be constructed using `ULA_MAKE_ATTR` or similar macros.
- After `proc_new`, you may call `proc_setflags` to adjust resource flags before starting.
- The child process (for "s") inherits the parent's memory image at the time of `proc_new`, but does not execute until `proc_start`.

5.1.6 Changelog

Version	Change
1.0	Initial design: <code>proc_new</code> only supported "n" mode.
3.0	Added "s" mode for clone (fork) semantics, which ran immediately.
10.1	Removed <code>flags</code> parameter; "s" mode now also requires <code>proc_start</code> . Added <code>proc_setflags</code> for separate flag configuration.

5.2 `proc_setflags`

```
int proc_setflags(pid_t pid, uint32_t flags);
```

5.2.1 Description

Sets or modifies the resource flags for a process created with `proc_new`. This must be called before `proc_start`. After the process has started, flags cannot be changed.

5.2.2 Parameters

pid	pid_t	PID of the process.
flags	uint32_t	Bitmask of resource flags (e.g., <code>PROC_IO_READ</code>).

5.2.3 Return Value

0 on success, negative error code otherwise (`-ESRCH`, `-EINVAL`, `-EPERM`).

5.2.4 Design Rationale

Separating flag configuration from creation allows the caller to adjust permissions based on runtime conditions. This is more flexible than fixing flags at creation time.

5.2.5 Changelog

Introduced in 10.1.

5.3 `proc_start`

```
int proc_start(pid_t pid, const char *path, const parg_t *parg);
```

5.3.1 Description

Starts a process created with `proc_new`. Loads the executable at `path` and passes `argc/argv/envp`. The process must not have been started before. Returns 0 on success, negative error code otherwise.

5.3.2 Parameters

pid	pid_t	PID of the process.
path	const char *	Path to the executable file.
parg	const parg_t *	Arguments and environment.

5.3.3 Return Value

0 on success, negative error code (-ENOENT, -ENOEXEC, -EINVAL).

5.4 `proc_terminate`

```
_Noreturn void proc_terminate(void);
```

Terminates the calling process immediately. Does not return.

5.5 `proc_kill`

```
int proc_kill(pid_t pid);
```

Sends `PROCSIG_KILL` to the process. Requires appropriate privilege. Returns 0 on success, negative error code.

5.6 `proc_signal_send`

```
int proc_signal_send(pid_t pid, uint32_t signum);
```

Sends a signal (e.g., `PROCSIG_TERM`) to a process. Returns 0 on success, negative error code.

5.7 Orphan Process Handling

When a process terminates, all its children are also terminated (they are not reparented to `init`). To keep a child alive after parent exits, the parent must have installed a signal handler for `PROCSIG_CHLD` and the child must have been marked via an implementation-defined mechanism. Default: children are killed.

5.8 `proc_getpid`, `proc_getppid`

```
pid_t proc_getpid(void);  
pid_t proc_getppid(void);
```

Return PID and parent PID of the caller.

5.9 `waitpid`

```
pid_t waitpid(pid_t pid, int *status, int options);
```

Only `WNOHANG` supported. Returns child PID, 0 if `WNOHANG` and none exited, or `-ECHILD`.

5.10 `whoami`

```
int whoami(void);
```

Returns the current ULA level (numeric).

5.11 set_privilege

```
int set_privilege(uint32_t mask);
```

Requires hlud authorization. Sets process privilege mask.

6 Memory Management

6.1 page_malloc

```
void *page_malloc(uint32_t count);
```

6.1.1 Description

Allocates `count` contiguous physical pages (4096 bytes each), zero-filled. Returns virtual address or NULL. Requires PROC_MEMORY flag.

6.1.2 Parameters

count	uint32_t	Number of pages (must be ≥ 0).
-------	----------	--------------------------------------

6.1.3 Return Value

Success: non-NULL virtual address. Failure: NULL.

6.1.4 Design Rationale

Lowest-level memory primitive for DMA and page allocators. Zero-fill prevents information leaks.

6.1.5 Changelog

1.0 introduced; 5.0 added PROC_MEMORY requirement; 10.2 renamed to `page_malloc/page_mfree`.

6.2 page_mfree

```
void page_mfree(void *pages, uint32_t count);
```

Frees pages allocated with `page_malloc`. Behaviour undefined if parameters mismatch.

6.3 shm_open

```
fid_t shm_open(const char *name, int oflags, mode_t mode);
```

Creates/opens a shared memory object. Returns `fid_t` or negative error.

6.4 shm_unlink

```
int shm_unlink(const char *name);
```

Removes a shared memory object. Returns 0 on success, negative error.

7 File and Device I/O

7.1 open

```
fid_t open(const char *type, const char *mode, const char *addr);
```

7.1.1 Description

Opens a regular file (`type = "f"`) or a device (`type = "d"`). For network I/O, see `net_*` functions (Section 9). Returns a file descriptor (`fid_t`) that can be used with `read`, `write`, `io_seek`, and `close`.

7.1.2 Parameters

Parameter	Type	Description
<code>type</code>	<code>const char *</code>	"f" (file) or "d" (device).
<code>mode</code>	<code>const char *</code>	"r" (read only), "w" (write only, truncate/create), "rw" (read/write).
<code>addr</code>	<code>const char *</code>	Path (absolute or relative) or device name (e.g., <code>/dev/tty0</code> or <code>"tty0"</code> interpreted under <code>/dev</code>).

7.1.3 Return Value

Success: non-negative `fid_t`. Failure: negative error code (`-ENOENT`, `-EPERM`, `-EINVAL`, `-EMFILE`).

7.1.4 Design Rationale

Uniform handling of files and devices simplifies the API. Network I/O is kept separate because its semantics differ significantly (no offsets, accept operation). The `type` string is used for type safety in the C library wrapper.

7.1.5 Notes for Callers

- Requires `PROC_IO_READ` for reading, `PROC_IO_WRITE` for writing.
- "w" mode truncates the file to zero length. Use "rw" to preserve content.

7.1.6 Changelog

1.0	Introduced.
5.0	Added ULA permission checks.
6.0	Removed network type "n" (moved to <code>net_connect</code>).
10.1	Added complete documentation.

7.2 close

```
int close(const char *type, fid_t fid);
```

Closes the I/O object. `type` must match.

7.3 read

```
ssize_t read(const char *type, fid_t fid, void *buf, size_t count);
```

Reads up to count bytes from file/device/pipe. Returns bytes read, 0 on EOF, or negative error.

7.4 write

```
ssize_t write(const char *type, fid_t fid, const void *buf, size_t count);
```

Writes up to count bytes. Returns bytes written or negative error.

7.5 io_seek

```
off_t io_seek(const char *type, fid_t fid, off_t offset, int whence);
```

Repositions file offset. whence = SEEK_SET, SEEK_CUR, SEEK_END. Returns new offset or negative error.

7.6 Working Directory Functions

```
int godir(const char *path);
```

Changes the current working directory to path. Returns 0 on success or negative error.

```
int godir_byfid(const char *type, fid_t fid);
```

Changes the current working directory to the directory matching the open fid_t. Returns 0 on success or negative error.

```
int getwd(char *buf, size_t size);
```

Copies the absolute path of the current working directory into buf up to size bytes. Returns 0 on success or negative error.

7.7 pipe

```
fid_t pipe(pid_t to_who);
```

Creates a unidirectional pipe. Write end returned; read end automatically transferred to to_who. Type is "p".

7.8 cpid, cpid2, set_flags, umask

Standard descriptor duplication and flag functions.

8 Extended File System Operations (Added in 10.3)

These operations reside in `<slfd/fs.h>` and interact directly with the file system server.

8.1 Metadata and Attributes

```
int stat(const char *path, struct stat *buf);
```

Retrieves information about the file pointed to by `path` and populates the `stat` struct.

8.2 Directory Creation and Removal

```
int mkdir(const char *path, uint32_t attr_mask);
```

Creates a directory named `path` with the specified ULA attribute mask.

```
int rmdir(const char *path);
```

Removes the directory `path` (must be empty).

8.3 Directory Traversal

```
fid_t fs_opendir(const char *path);
```

Opens a directory stream and returns a directory descriptor.

```
int fs_readdir(fid_t dir, struct fs_dirent *ent);
```

Reads the next entry from the directory descriptor into `ent`. Returns 1 on successful read, 0 on end of directory, negative on error.

```
int fs_closedir(fid_t dir);
```

Closes the directory stream.

8.4 File Manipulation

```
int fs_unlink(const char *path);
```

Removes the file or symbolic link at `path`.

```
int fs_rename(const char *oldpath, const char *newpath);
```

Moves or renames a file from `oldpath` to `newpath`.

9 Network Communications

Network I/O is handled separately from file I/O. All functions require the `PROC_NETWORK` flag.

9.1 net_connect

```
fid_t net_connect(const char *protocol, const char *address);
```

9.1.1 Description

Establishes a connection to a remote host. `protocol` is "tcp" (future: "udp"). `address` is "ip:port" (e.g., "192.168.1.1:80").

9.1.2 Parameters

<code>protocol</code>	<code>const char *</code>	"tcp" (or "udp" in future).
<code>address</code>	<code>const char *</code>	e.g., "192.168.1.1:80" or "[::1]:8080".

9.1.3 Return Value

Success: `fid_t`. Failure: negative error (-ECONNREFUSED, -EINVAL, -ENETUNREACH).

9.1.4 Design Rationale

Separate from file I/O to avoid confusion (no file offsets, accept semantics). Cleaner API for network-specific operations.

9.1.5 Changelog

6.0 introduced; 10.1 added complete documentation.

9.2 net_listen

```
fid_t net_listen(const char *protocol, const char *address);
```

Creates a listening socket. Returns `fid_t` or negative error (-EADDRINUSE, -EINVAL).

9.3 net_accept

```
fid_t net_accept(fid_t listen_fid);
```

Accepts an incoming connection. Returns new `fid_t` or negative error.

9.4 net_send

```
ssize_t net_send(fid_t fid, const void *buf, size_t count);
```

Sends data on a connected socket. Returns bytes sent or negative error (-EPIPE, -ENOTCONN).

9.5 net_recv

```
ssize_t net_recv(fid_t fid, void *buf, size_t count);
```

Receives data from a connected socket. Returns bytes received, 0 on remote close, or negative error.

9.6 net_disconnect

```
int net_disconnect(fid_t fid);
```

Closes the network connection. Returns 0 on success, negative error.

10 Event Multiplexing (hear Interface)

The **hear** interface allows waiting for events from multiple sources (I/O, IPC, signals, process state).

10.1 hear_add

```
int hear_add(fid_t fid, int events);
```

10.1.1 Description

Adds **fid** to the monitored set. **events** is a bitmask of **HEAR_IN**, **HEAR_OUT**, **HEAR_IPC**, **HEAR_SIGNAL**, **HEAR_PROC**, etc.

10.1.2 Parameters

fid	fid_t	File/pipe/socket/process to monitor.
events	int	Bitmask of event types.

10.1.3 Return Value

0 on success, negative error otherwise.

10.1.4 Design Rationale

Unified event notification simplifies application design. One interface for all async events.

10.1.5 Changelog

6.0 introduced; 10.1 added complete documentation.

10.2 hear_remove

```
int hear_remove(fid_t fid);
```

Removes **fid** from the monitored set.

10.3 hear_filter

```
int hear_filter(fid_t fid, int new_events);
```

Changes the event mask for an existing source.

10.4 hear_wait

```
int hear_wait(void *events_out, int max_events, int timeout_ms);
```

10.4.1 Description

Waits for events among registered sources. Fills `events_out` with up to `max_events` event structures. Each event contains `handle`, `type`, and `userdata` (if set).

10.4.2 Parameters

<code>events_out</code>	<code>void *</code>	Pointer to array of <code>struct hear_event</code> .
<code>max_events</code>	<code>int</code>	Maximum number of events to return.
<code>timeout_ms</code>	<code>int</code>	Timeout in milliseconds (0 = non-blocking, -1 = infinite).

10.4.3 Return Value

Number of events received, 0 on timeout, negative error on failure.

10.4.4 Example

```
fid_t fd1 = open("f", "r", "/tmp/a");
fid_t fd2 = open("f", "r", "/tmp/b");
hear_add(fd1, HEAR_IN, NULL);
hear_add(fd2, HEAR_IN, NULL);
struct hear_event ev[4];
int n = hear_wait(ev, 4, 1000);
for (int i = 0; i < n; i++) {
    if (ev[i].handle == fd1) read("f", fd1, ...);
    else if (ev[i].handle == fd2) read("f", fd2, ...);
}
```

11 Inter-Process Communication (IPC)

SLFD provides synchronous message passing between threads. Each thread has a kernel-managed message queue.

11.1 System Calls

```
/* It's used for 'syscall()', for user space process, please use
   ipc_send_msg */
int sys_ipc_send(thread_id target, ipc_message_t *msg);
int sys_ipc_receive(ipc_message_t *msg);
```

11.1.1 Description

`sys_ipc_send` sends `msg` to the target thread. The kernel fills `source` and `timestamp`. If the target's queue is full, the caller may block (implementation-defined). `sys_ipc_receive` retrieves the next message from the caller's queue. If no message is available, the call blocks until one arrives.

11.1.2 Return Value

0 on success, negative error code otherwise.

11.2 C Library Wrappers

```
int ipc_send_msg(uint32_t type, uint32_t flags, uint32_t
    receiver_pid,
                uint32_t data_len, const void *data);
int ipc_receive_msg(ipc_message_t *msg, int timeout_ms);
```

Wrapper functions for convenience. `ipc_send_msg` sends to the main thread of the target process. `ipc_receive_msg` supports timeout (0 = non-blocking, -1 = infinite).

11.3 Error Codes for IPC

-EAGAIN, -ETIMEDOUT, -EMSGSIZE, -ENOMSG.

11.4 IPC Performance Tuning (Implementation-Defined)

```
int ipc_set_buffer(thread_id target, void *buffer, size_t size);
int ipc_use_buffer(thread_id target, void *buffer, size_t size);
```

Optional functions for shared memory message caching. May be no-ops.

12 Thread Management

12.1 thread_create

```
int thread_create(thread_id *tid, void *(*start)(void*), void *
    arg);
```

Creates a new thread that starts executing `start(arg)`. Inherits signal mask and ULA privilege. Returns 0 on success.

12.2 thread_join

```
int thread_join(thread_id tid, void **retval);
```

Waits for thread termination. Stores exit value in `retval`.

12.3 thread_exit

```
_Noreturn void thread_exit(void *retval);
```

Terminates the calling thread.

12.4 Mutexes and Condition Variables

Standard POSIX-style primitives:

```
typedef struct mutex { /* opaque */ } mutex_t;
int mutex_init(mutex_t *mutex);
int mutex_lock(mutex_t *mutex);
int mutex_unlock(mutex_t *mutex);
int mutex_destroy(mutex_t *mutex);

typedef struct cond { /* opaque */ } cond_t;
int cond_init(cond_t *cond);
int cond_wait(cond_t *cond, mutex_t *mutex);
int cond_signal(cond_t *cond);
int cond_broadcast(cond_t *cond);
int cond_destroy(cond_t *cond);
```

All return 0 on success, negative error otherwise.

13 Error Codes

Full list of error codes (as in version 9.0). Include all codes with descriptions. I'll provide the complete table (longtable).

Symbolic Name	Value	Description
-EPERM	1	Permission denied.
-ENOENT	2	No such file or directory.
-ESRCH	3	No such process.
-EINTR	4	Interrupted system call.
-EIO	5	I/O error.
-ENXIO	6	No such device or address.
-E2BIG	7	Argument list too long.
-ENOEXEC	8	Exec format error.
-EBADF	9	Bad file descriptor.
-ECHILD	10	No child process.
-EAGAIN	11	Resource temporarily unavailable.
-ENOMEM	12	Out of memory.
-EACCES	13	Permission denied (file access).
-EFAULT	14	Bad address.
-ENOTBLK	15	Block device required.
-EBUSY	16	Device or resource busy.
-EEXIST	17	File exists.
-EXDEV	18	Cross-device link.
-ENODEV	19	No such device.
-ENOTDIR	20	Not a directory.
-EISDIR	21	Is a directory.
-EINVAL	22	Invalid argument.
-ENFILE	23	File table overflow.
-EMFILE	24	Too many open files.

Continued on next page

<i>Continued from previous page</i>		
Symbolic Name	Value	Description
-ENOTTY	25	Inappropriate I/O control.
-ETXTBSY	26	Text file busy.
-EFBIG	27	File too large.
-ENOSPC	28	No space left on device.
-ESPIPE	29	Illegal seek.
-EROFS	30	Read-only file system.
-EMLINK	31	Too many links.
-EPIPE	32	Broken pipe.
-EDOM	33	Math argument out of domain.
-ERANGE	34	Result too large.
-EDEADLK	35	Deadlock avoided.
-ENAMETOOLONG	36	File name too long.
-ENOLCK	37	No locks available.
-ENOSYS	38	Function not implemented.
-ENOTEMPTY	39	Directory not empty.
-ELOOP	40	Too many symbolic links.
-EWOULDBLOCK	41	Same as -EAGAIN.
-ENOMSG	42	No message of desired type.
-EIDRM	43	Identifier removed.
-ETIMEDOUT	44	Connection timed out.
-ERESTART	45	Restart syscall (internal).
-EMSGSIZE	46	Message too long.
-EADDRINUSE	47	Address already in use.
-ECONNREFUSED	48	Connection refused.
-ECONNRESET	49	Connection reset by peer.
-ENOTCONN	50	Socket not connected.
-ETIME	51	Timer expired.
-ECANCELED	52	Operation canceled.

A ULA: User Level Assessment

This appendix defines the ULA privilege model. It replaces traditional Unix UID/GID and mode_t with a level-based bitmask system.

A.1 Privilege Levels

```
#define LV_HIGHEST    0
#define LV_GENERAL    1
#define LV_LOW        2
#define LV_GETTING    3
```

A.2 File Attribute Mask Layout

The 32-bit mask:

- Bits 31–24: creator level

- Bits 23–16: caller level
- Bits 15–8: flags (EXEC, HIDDEN, ARCHIVE)
- Bits 7–4: file type (FILE, DIR, LINK, DEV)
- Bits 3–0: reserved

A.3 Access Control Matrix

Operation	LV	LV_HIGHEST	LV_LOW	LV_GETTING
<code>proc_kill</code> (any process)	Yes	No	No	No
<code>proc_kill</code> (own process)	Yes	Yes	Yes	Yes
<code>proc_new</code> ("s", ...) (clone)	Yes	Yes	No	No
<code>proc_signal_send</code> (any)	Yes	No	No	No
<code>open</code> ("d", ...) (any device)	Yes	Limited ^a	No	Limited ^b
<code>open</code> ("f", "w") on system dirs	Yes	No	No	No
<code>open</code> ("f", "r") on system files	Yes	No ^c	No	Yes ^d
<code>proc_new</code> ("n", ...)	Yes	Yes	Yes	Yes
<code>getwd</code> , <code>stat</code> , <code>mkdir</code>	Yes	Own files only	Own files only (restricted)	Own work dir ^e

Notes:

1. LV_GENERAL may open non-sensitive devices (e.g., `/dev/tty`) but not raw block devices.
2. LV_GETTING may open devices required for its service (e.g., `/dev/null`) but not console.
3. LV_GENERAL cannot read system config files (e.g., `/conf/shadow`).
4. LV_GETTING may read specific system files as defined by service manifest.
5. LV_GETTING uses service-specific working directory (e.g., `/usr/myapp`) and cannot access others' home.

A.4 Textual Database Example

```
# /conf/users
alice:{staff-lvgeneral}+{audio-lvlow}
bob:{staff-lvlow}
root:{wheel-lvhighest}
# /conf/groups
staff:500:alice,bob
wheel:0:root
audio:300:alice
```

A.5 Changelog

1.0	Initial design.
3.0	Added access control matrix.
5.0	Bitmask layout introduced.
7.0	Added textual database example.
9.0	Added rationale.
10.1	Moved to Appendix A.

B SLFD Free Desktop (SLFD FD)

This appendix defines the root directory layout for SLFD-based systems.

B.1 Root Directory Structure

`/bin` Essential system binaries (e.g., `sh`, `ls`).

`/dev` Device special files.

`/conf` System and application configuration files (replaces `/etc`).

`/usr` Combined user and system resources (user home directories + system binaries/libraries).

`/lib` Common libraries (may be symlink to `/usr/lib`).

`/tmp` Temporary files (cleared on reboot).

`/sys` System files (e.g., `/sys/kernel/debug`).

`/mnt` Mount point for devices.

B.2 Rationale

Avoids FHS and freedesktop.org complexity. All configuration in `/conf`, all data under `/usr`, essential binaries in `/bin`.

B.3 Example

```
/
+-- bin/
|   +-- sh
|   +-- ls
+-- dev/
|   +-- tty0
|   +-- random
|   +-- null
+-- conf/
|   +-- passwd
|   +-- shadow
|   +-- slfd.conf
|   +-- wahlu
```

```

+-- usr/
|   +-- bin/      (optional symlink to /bin)
|   +-- lib/
|   +-- alice/
|   +-- bob/
|   +-- ...
+-- mnt/
|   +-- dev1/
|   +-- dev2/
|   +-- ...
+-- sys/
|   +-- ...
+-- lib/          (optional symlink to usr/lib)
+-- tmp/

```

B.4 Changelog

1.0	Initial: /bin, /dev, /etc, /usr, /lib, /tmp.
2.0	Replaced /etc with /conf.
3.0	Added /sys, /mnt.
5.0	Renamed to SLFD FD.
10.1	Moved to Appendix B.

C SLFD Shell Standard (3S)

Non-mandatory shell standard. Commands:

Command	Description
lookup [path]	List directory contents.
goto <path>	Change directory.
where	Print current directory.
create.dir <path>	Create directory.
del.dir <path>	Remove empty directory.
del <path>	Remove file/link.
connect <file...>	Concatenate files to stdout.
count [options] [file]	Count lines/words/bytes.
head [-n N] [file]	Output first N lines.
tail [-n N] [file]	Output last N lines.
filter <pattern> [file...]	Search for pattern.
pm [options]	Process management.
pm kill <PID> [signal]	Send signal.
env [var[=value]...]	Environment.
sysinfo	System information.
info-of <command>	Manual page.

Extended ULA commands: ula file-change, ula user-change.

C.1 Non-mandatory Status

This standard is optional. A shell may claim 3S compliance if it implements these commands.

C.2 Changelog

1.0	Initial.
4.0	Updated to match SLFD FD naming.
5.0	Added <code>pm</code> , <code>pm kill</code> .
10.1	Moved to Appendix C.

D Kernel Primitives and System Call Interfaces

This appendix lists mandatory and optional kernel primitives.

D.1 Mandatory Kernel Primitives

Following the MINIX 3 model, the kernel itself provides only the minimal primitives needed to manipulate process control blocks (PCBs) and pass messages. Higher-level process management policy (validation, sequencing, signal delivery semantics) is implemented by a user-space process server; see §D.2.

Primitive	Description
<code>sys_pcb_alloc</code>	Allocate a PCB slot and PID; does not load or start execution. Used internally by the process server.
<code>sys_pcb_free</code>	Release a PCB slot. Used internally by the process server.
<code>sys_mm_alloc_pages</code>	Allocate physical pages (backing <code>page_malloc</code>).
<code>sys_mm_free_pages</code>	Free pages (backing <code>page_mfree</code>).
<code>sys_shm_open</code>	Open shared memory.
<code>sys_shm_unlink</code>	Remove shared memory.
<code>sys_ipc_send</code>	Send IPC message.
<code>sys_ipc_receive</code>	Receive IPC message.
<code>sys_getrlimit</code>	Get resource limits.
<code>sys_setrlimit</code>	Set resource limits.
<code>sys_sleep_pro</code>	Sleep for seconds.
<code>sys_timer_crate</code>	Create timer.
<code>sys_get_fucktimer</code>	Get monotonic time.
<code>sys_settimeofday</code>	Set real-time clock.
<code>sys_info_sys</code>	Fill <code>utsname</code> .
<code>sys_getrandom</code>	Fill buffer with random.
<code>sys_hlud_request</code>	Request elevation.

D.2 Optional Primitives

These may be implemented as user-space servers.

Function	Typical Implementation
----------	------------------------

proc_new, proc_setflags, proc_start, proc_terminate, proc_kill, proc_signal_send, proc_getpid, proc_getppid, waitpid, whoami, set_privilege	Process management server (PM-equivalent, per the MINIX 3 model)...
proc_new, proc_setflags, proc_start, proc_terminate, proc_kill, proc_signal_send, proc_getpid, proc_getppid, waitpid, whoami, set_privilege	Process management server (PM-equivalent, per the MINIX 3 model). Implements all argument validation, ULA privilege checks, and multi-step sequencing; internally uses <code>sys_pcb_alloc/sys_pcb_free</code> for the actual kernel-level PCB manipulation. Clients reach this server over IPC, never via direct trap.
open, close, read, write, io_seek	File system server.
godir, godir_byfid, getwd	File system server.
pipe	IPC or kernel service.
cpfid, cpfid2	Kernel or server.
set_flags	Kernel.
umask	User-space library.
dev_control	Device driver server.
chmod, chown, stat	File system server.
mkdir, rmdir	File system server.
symlink, readlink	File system server.
mount, umount	File system server.
reboot	System management server.
fs_*	File system server.
thread_*, mutex/cond	Library or kernel-assisted.
cp0_*, aio_*	Implementation-defined.
ipc_set_buffer, ipc_use_buffer	Implementation-defined.
hear_*	System call or event server.
net_*	Network server.

D.3 Changelog

1.0	Initial.
3.0	Added <code>sys_proc_signal_send</code> .
5.0	Added ULA primitives.
6.0	Added <code>hear</code> and network.
7.0	Split into mandatory/optional.
10.1	Corrected names, moved to Appendix D.
10.2	Adopted MINIX 3 model: replaced monolithic <code>sys_proc_*</code> kernel primitives with minimal <code>sys_pcb_alloc/sys_pcb_free</code> ; process management (<code>proc_*</code> , <code>waitpid</code> , <code>whoami</code> , <code>set_privilege</code>) moved to Optional Primitives as a user-space process server.